

Digital Nurture DSA Assignment

Exercise 1: Inventory Management System

1. Understand the Problem

Large inventories contain thousands of products. Efficient data structures reduce the time needed to search, update and delete products. A HashMap is suitable because products can be accessed directly using the product ID.

2. Setup

Create a Java project named InventoryManagementSystem. Create two classes: Product and InventoryManagement.

Product.java

```
class Product{
    int productId; String productName; int quantity; double price;
    Product(int id,String name,int qty,double price){
        this.productId=id; this.productName=name;
        this.quantity=qty; this.price=price;
    }
}
```

InventoryManagement.java

```
import java.util.*;
class InventoryManagement{
    HashMap<Integer,Product> inventory=new HashMap<>();
    void addProduct(Product p){inventory.put(p.productId,p);}
    void updateProduct(int id,int qty){
        if(inventory.containsKey(id))
            inventory.get(id).quantity=qty;
    }
    void deleteProduct(int id){inventory.remove(id);}
    public static void main(String[] args){
        InventoryManagement im=new InventoryManagement();
        im.addProduct(new Product(101,"Laptop",10,55000));
        im.updateProduct(101,20);
        im.deleteProduct(101);
    }
}
```

Analysis

Operation	Complexity
Add	O(1)
Update	O(1)

Delete

$O(1)$

Optimization: HashMap provides constant-time average performance using productId as the key.

Exercise 2: E-commerce Platform Search Function

1. Theory

Big O notation describes how the running time of an algorithm grows with the input size. Linear Search checks every element, while Binary Search repeatedly divides a sorted array into halves.

Product.java

```
class Product{
    int productId;
    String productName;
    String category;
    Product(int id,String name,String category){
        this.productId=id;
        this.productName=name;
        this.category=category;
    }
}
```

Linear Search

```
static Product linearSearch(Product[] arr,String key){
    for(Product p:arr)
        if(p.productName.equals(key))
            return p;
    return null;
}
```

Binary Search

```
static int binarySearch(Product[] arr,String key){
    int low=0,high=arr.length-1;
    while(low<=high){
        int mid=(low+high)/2;
        int cmp=arr[mid].productName.compareTo(key);
        if(cmp==0) return mid;
        if(cmp<0) low=mid+1;
        else high=mid-1;
    }
    return -1;
}
```

Algorithm

Time Complexity

Linear Search

$O(n)$

Binary Search

$O(\log n)$

Binary Search is more suitable for large datasets because it significantly reduces the number of comparisons, provided the array is sorted.

Exercise 3: Sorting Customer Orders

1. Understand Sorting Algorithms

Sorting algorithms arrange data in a specific order. In an e-commerce platform, sorting customer orders by total price helps prioritize high-value orders.

Bubble Sort:

- Compares adjacent elements and swaps them if needed.
- Simple but inefficient for large datasets.

Insertion Sort:

- Inserts each element into its correct position.
- Efficient for small or nearly sorted data.

Quick Sort:

- Uses Divide and Conquer.
- Fast for large datasets.

Merge Sort:

- Divides, sorts and merges data.
- Stable and efficient but requires extra memory.

2. Setup

Create a Java project named OrderSortingSystem with two classes: Order and SortingDemo.

3. Implementation

Order.java

```
class Order {
    int orderId;
    String customerName;
    double totalPrice;

    Order(int orderId, String customerName, double totalPrice){
        this.orderId = orderId;
        this.customerName = customerName;
        this.totalPrice = totalPrice;
    }
}
```

Bubble Sort

```
public static void bubbleSort(Order[] orders){
    int n = orders.length;
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-i-1;j++){
            if(orders[j].totalPrice > orders[j+1].totalPrice){
                Order temp = orders[j];
                orders[j]=orders[j+1];
                orders[j+1]=temp;
            }
        }
    }
}
```

Quick Sort

```
public static void quickSort(Order[] orders,int low,int high){
    if(low<high){
        int pivot=partition(orders,low,high);
        quickSort(orders,low,pivot-1);
        quickSort(orders,pivot+1,high);
    }
}
```

4. Analysis

Algorithm	Time Complexity
Bubble Sort	$O(n^2)$
Quick Sort (Average)	$O(n \log n)$
Quick Sort (Worst)	$O(n^2)$

Quick Sort is generally preferred because it is much faster than Bubble Sort for large datasets.

Exercise 4: Employee Management System

1. Understand Array Representation

Arrays store elements in contiguous memory locations. They provide fast index-based access and are easy to implement.

2. Setup

Create a Java project named EmployeeManagementSystem with Employee and EmployeeManagement classes.

3. Implementation

```
class Employee{
    int employeeId;
    String name;
    String position;
    double salary;

    Employee(int employeeId,String name,String position,double salary){
        this.employeeId=employeeId;
        this.name=name;
        this.position=position;
        this.salary=salary;
    }
}

class EmployeeManagement{
    Employee[] employees=new Employee[10];
    int size=0;

    void addEmployee(Employee e){ employees[size++]=e; }

    void searchEmployee(int id){
        for(int i=0;i<size;i++)
            if(employees[i].employeeId==id){
                System.out.println("Employee Found");
                return;
            }
        System.out.println("Employee Not Found");
    }

    void traverseEmployees(){
        for(int i=0;i<size;i++)
            System.out.println(employees[i].name);
    }

    void deleteEmployee(int id){
        for(int i=0;i<size;i++){
            if(employees[i].employeeId==id){
                for(int j=i;j<size-1;j++)
                    employees[j]=employees[j+1];
                size--;
                break;
            }
        }
    }
}
```

```

    }
  }
}
}

```

4. Analysis

Operation	Time Complexity
Add	$O(1)$
Search	$O(n)$
Traverse	$O(n)$
Delete	$O(n)$

Arrays are suitable when the number of records is fixed. For dynamic data, ArrayList or LinkedList is a better choice.

Exercise 5: Task Management System

1. Understand Linked Lists

A linked list is a linear data structure where each node stores data and a reference to the next node. Singly linked lists have one pointer, while doubly linked lists have both previous and next pointers.

2. Setup

Create a Java project named TaskManagementSystem with Task and TaskList classes.

3. Implementation

```

class Task{
    int taskId; String taskName,status; Task next;
    Task(int id,String name,String status){
        this.taskId=id; this.taskName=name; this.status=status;
    }
}

class TaskList{
    Task head;
    void add(Task t){
        if(head==null){head=t;return;}
        Task cur=head; while(cur.next!=null) cur=cur.next;
        cur.next=t;
    }
    void traverse(){
        Task cur=head;
        while(cur!=null){
            System.out.println(cur.taskName);
            cur=cur.next;
        }
    }
}

```

```

    }
}
}

```

4. Analysis

Operation	Time Complexity
Add	$O(n)$
Search	$O(n)$
Traverse	$O(n)$
Delete	$O(n)$

Exercise 6: Library Management System

1. Search Algorithms

Linear Search checks every book one by one. Binary Search repeatedly divides a sorted array into halves.

2. Setup

Create Book and Library classes.

3. Implementation

```

class Book{
    int bookId; String title,author;
    Book(int id,String t,String a){
        bookId=id; title=t; author=a;
    }
}

static Book linearSearch(Book[] books,String key){
    for(Book b:books)
        if(b.title.equals(key)) return b;
    return null;
}

static int binarySearch(Book[] books,String key){
    int low=0,high=books.length-1;
    while(low<=high){
        int mid=(low+high)/2;
        int c=books[mid].title.compareTo(key);
        if(c==0) return mid;
        if(c<0) low=mid+1; else high=mid-1;
    }
    return -1;
}

```

4. Analysis

Algorithm	Time Complexity
Linear Search	$O(n)$
Binary Search	$O(\log n)$

Exercise 7: Financial Forecasting

1. Recursion

Recursion is a technique in which a method calls itself until a base condition is reached.

2. Setup

Create a Java project with a Forecast class.

3. Implementation

```
public class Forecast{
    static double futureValue(double value,int years,double rate){
        if(years==0) return value;
        return futureValue(value,years-1,rate)*(1+rate);
    }
    public static void main(String[] args){
        System.out.println(futureValue(10000,5,0.10));
    }
}
```

4. Analysis

Time Complexity: $O(n)$

Space Complexity: $O(n)$ due to recursion.

Optimization: Memoization or an iterative solution can reduce repeated computations and improve space usage.